AURORA LIGHT • BLE CONTROLLER



幻彩灯控
使用与协议文档

无注册 · 无登录 · 仅联网检查更新

BLE 幻彩灯控制 App v1.0.9

Android 6.0+ · BLE 4.0+

文档版本 1.0 · 2026-11

界面预览

主界面采用深色极光、玻璃卡片和高对比渐变控件；下图为与实现共用同一视觉规范的设计预览。实际 Android 渲染会因机型、字体和状态栏高度略有差异。



主界面 · 设计预览



连接面板 · 设计预览

幻彩灯控 · 使用手册

版本：v1.0.9
适用系统：Android 6.0 及以上（需支持低功耗蓝牙 BLE 4.0+）
App 名称：幻彩灯控（Aurora Light）

---

1. 这是什么

一个**不需要注册、不需要登录**的蓝牙幻彩灯控制 App；仅联网检查 GitHub 更新和下载新版本。打开后自动连接上次设备，也可以扫描新设备，控制开关、亮度、颜色、光效和速度。

App 只做三件事：

1. 通过低功耗蓝牙（BLE）发现附近的幻彩灯；
2. 向灯带发送控制指令；
3. 接收灯带回报的状态，保持界面同步。

所有设置保存在手机本地，不会上传到任何服务器。

---

2. 30 秒上手

1. 给灯带通电，确认灯带处于可被蓝牙发现的状态。
2. 安装 `AuroraLight-v1.0.9.apk`（首次安装需允许“安装未知来源应用”）。
3. 打开 App。首次使用会弹出系统权限请求，请允许“附近的设备”；Android 11 及以下请允许定位权限并打开系统定位开关。
4. 点底部的 **“连接灯光”**，或点右上角状态胶囊。
5. 在底部面板中选择名称以 `AuroraLight-` 开头的设备。
6. 连接成功后，App 会自动把当前开关、亮度、颜色、光效同步给灯带。

> 没有灯带也可以先点 **“没有设备？点此体验演示模式”**，完整预览界面和操作。演示模式不会发送真实蓝牙指令。

---

3. 界面说明

3.1 顶部状态

显示	含义
灰色 · 未连接	尚未连接灯带，可以预先调好设置，连接后自动同步
紫色 · 连接中	正在建立蓝牙连接
绿色 · 已连接	已连接并可以发送指令
琥珀色 · 演示模式	当前操作只在本机模拟

点状态胶囊：未连接时打开扫描面板；已连接时显示当前设备名。

3.2 主开关

中央的大圆形按钮是总开关。

- 轻触：开灯 / 关灯。
- 开启时外圈有呼吸光晕，按钮颜色会跟随当前灯光颜色变化。
- **未连接蓝牙时不能切换**，会提示“请先连接蓝牙，再开关灯光”。

3.3 亮度

拖动滑块，顶部右侧显示 0–100% 的数值。**拖动过程中每约 120ms 发送一次当前亮度**，松手后再补发 3 次，避免只在松手时发码。

3.4 光效模式

模式	说明
流光跑马	颜色沿灯带流动（协议跑马灯）
呼吸	亮度缓慢起伏
律动	跟随节拍闪烁（协议律动）
火焰	暖色随机闪烁（协议火焰）

点卡片即可切换。被选中的卡片有薄荷色描边和图标高亮。

3.5 律动速度

“慢 / 标准 / 快”三个档位，对应协议中的速度值 30 / 60 / 90。只对动态光效明显。

3.6 灯光颜色

九个协议颜色，对应协议颜色码 1–9：

- 红、橙、黄、绿、青、蓝、紫、白、品红

点颜色会自动切换到静态色，App 发送 `11 08 01 0n CS`（`n=1-9`）。

再次点击当前已高亮的颜色，会关闭颜色并发送：

```
11 08 01 00 1A
```

3.7 亮灯数量

“光效”页的亮灯数量：

- 默认 **1 个**；
- `-` / `+`：每次 1 颗，直接发送 `11 07`；
- 数值显示“当前 / 255”；
- 颜色只作用于当前亮灯数量；
- `0x07` 是相对命令，App 每次只发 1 次，避免重复 3 次后多加 3 颗。

3.8 自检

“光效”页的“自检 · 让灯珠跑一遍”：

- 点击发送 `11 FF 01 01 12`，按钮高亮；
- 再点发送 `11 FF 01 00 11`，取消高亮；
- 关灯、一键关闭、重新连接会自动取消自检。

3.9 自定义灯光

“光效”页的“自定义灯光 · 逐颗设置要亮的灯”进入全屏界面：

1. 先输入产品实际灯珠总数（1–255）；
2. 点“生成”，生成 1…N 的灯珠按钮；
3. 点灯珠点亮，再点一次关闭；可以同时点亮多颗；
4. 自定义界面顶部也有 9 色圆点，选色会作用于已经点亮的灯；
5. 快速点击会排队发送，上一组 3 次发完才发下一组，不会跳过；
6. 退出时发送 `11 12 02 00 00 25`。

单颗灯格式：

```
11 12 02 x y CS
```

例如点亮第 3 颗：`11 12 02 03 01 29`；关闭第 3 颗：`11 12 02 03 00 28`。

输入的总数在退出/再次进入时保留，直到重新连接蓝牙才清空；点“清除”可以手动重新输入。

3.10 开发者模式

进入“我的”页面，**长按版本号**：

- 长按进入开发者模式，再长按退出；
- 整个 App 仍然只有一个 Activity，不跳转新页面；
- 开发者模式下方会显示 App **实际发出的十六进制码**，字号较小，可滚动，右上角“清空”；
- App 界面不提供“长按进入”的文字提示。

每条控制命令会连续发送 **3 次，间隔 100ms**，所以日志里会看到同一条码重复 3 行。

3.11 一键关闭

第一页“一键关闭”按钮发送协议命令 `11 11 01 00 23`，同时：

- 总开关关闭；
- 所有模式高光熄灭；
- 亮度显示 0%；
- 颜色选中环熄灭。

3.12 连接 / 断开

底部主按钮会根据状态变化：

- **连接灯光**：打开扫描面板。
- **正在连接…**：轻触可取消连接。
- **已连接 • 轻触断开**：断开当前灯带。
- **演示模式 • 轻触退出**：退出演示。

3.13 自动连接

- 第一次通过扫描连接成功后，App 会保存设备地址；
- 后续启动 App，如果蓝牙和权限已开启，会直接用保存的地址连接；
- 不需要每次重新扫描；
- 自动连接失败时，仍可点“连接灯光”手动扫描；
- 自动连接成功后默认关灯，需要你自己按总开关。

3.14 自动更新

- App 启动后会检查 GitHub 上的 `update.json`；
- 有新版本会弹窗显示更新内容；
- 点“立即更新”下载 APK，校验 SHA-256 后拉起系统安装；
- 普通 App 无法静默安装，最后需要点一次系统“安装”；

- 详细配置见 `docs/自动更新说明.md`。

---

4. 连接与权限

4.1 Android 12 及以上

需要“附近的设备”权限（`BLUETOOTH_SCAN` + `BLUETOOTH_CONNECT`）。App 声明了 `neverForLocation`，不需要定位权限，也不会读取位置。

4.2 Android 6–11

系统限制：扫描 BLE 必须拥有定位权限，并且系统定位开关处于开启状态。App 不读取、不保存位置；定位权限只用于系统允许扫描蓝牙。

4.3 权限被拒绝

在扫描面板中：

- 点 **“重新授权”** 再次请求；
- 或点 **“权限被拒绝？打开系统设置”**，在系统设置里手动开启；
- 也可以先进入演示模式。

4.4 蓝牙未开启

面板会显示“蓝牙未开启”。点 **“开启蓝牙”** 后按系统提示操作，返回 App 会自动重新检测。

---

5. 设备兼容性（重要）

这个 App 使用**自定义 BLE 指令协议**，并使用中科蓝讯的 16 位 UUID：

- 服务 UUID：`0000BBB0-0000-1000-8000-00805F9B34FB`（`0xBBB0`）
- 控制特征值：`0000BBB1-0000-1000-8000-00805F9B34FB`（`0xBBB1`）
- 状态特征值：`0000BBB2-0000-1000-8000-00805F9B34FB`（`0xBBB2`）
- 只读信息：`0000BBB3-0000-1000-8000-00805F9B34FB`（`0xBBB3`，可选）
- 广播名称：`AuroraLight-XXXX`

App **只向** `0xBBB1` **写指令**，不读取 `0xBBB2/BBB3`，也不订阅 Notify；因此固件只要实现 `0xBBB1` 的写回调即可。

因此：

- **可以连接**：已经按本协议实现固件的灯带（参考项目内 ESP32 固件）。
- **不能直接连接**：市面上使用厂商私有协议、UUID 和指令都不同的成品灯带。App 会提示“该设备不是幻彩灯：未找到自定义服务”。

- 如果你的灯带是可编程控制器（ESP32 / nRF52 / Arduino + BLE 模块），可以烧录项目内的参考固件，或按《BLE 协议规范》适配现有固件。

> 换句话说：手机是“主机”，灯带是“从机”。只有灯带按约定广播并提供对应服务，App 才能控制它。

---

6. 常见问题

Q1：扫描不到灯带?

按顺序检查：

1. 灯带是否通电，是否处于可连接状态；
2. 手机蓝牙是否开启；
3. Android 12+ 是否允许“附近的设备”；Android 11 及以下是否允许定位且打开定位开关；
4. 灯带是否在 5 米内，中间是否有强干扰源；
5. 灯带固件是否真的广播了 `0000BBB0-0000-1000-8000-00805F9B34FB`；
6. 是否已被其他手机连接（BLE 从机通常只允许一个中心设备连接）；
7. 点“重新扫描”再试。

Q2：能连上，但提示“设备固件不完整”？

设备提供了自定义服务，但缺少控制或状态特征值。请核对固件里的三个 UUID 是否与协议完全一致，尤其是 `0002` / `0003` 这两段。

Q3：连接后点按钮没反应?

- 看顶部状态是否为绿色“已连接”；
- 看灯带固件是否实现了对应指令；
- 在开发者文档的“测试清单”中用 nRF Connect 手动写一帧，确认固件能响应；
- 若使用 iOS 或电脑调试，请确认写入方式为 **Write With Response**。

Q4：为什么 App 没有“登录 / 注册”？

这是刻意设计。没有账号体系，没有云端，没有广告 SDK。安装即用，所有偏好只写入本机 `SharedPreferences`。

Q5：换手机后设置还在吗?

不在。设置保存在本机，不联网同步；仅更新检查会访问 GitHub。重新连接后 App 会把当前界面状态同步给灯带。

Q6：卸载重装后签名冲突?

请先卸载旧版本再安装。不同签名的 APK 不能直接覆盖安装。

Q7：灯带颜色和 App 里选的不完全一样?

RGB 灯珠存在批次差异，且固件可能做了亮度/色温校正。可在固件里对 RGB 做校准，或选择更接近的颜色。

---

7. 隐私与安全

- **INTERNET 权限仅用于检查 GitHub 更新和下载 APK**，不上传个人信息。
- 没有账号、手机号、邮箱、通讯录、相册、位置读取逻辑。
- 不收集、不上传任何蓝牙数据或个人数据。
- 蓝牙连接不配对、不加密，适合个人近距离灯光控制；不要用于门锁等安全场景。
- 设置仅存储在本机，卸载 App 即删除。

---

8. 卸载与更新

- 更新：使用同一签名的新 APK 覆盖安装，设置会保留。
- 卸载：长按桌面图标 → 应用信息 → 卸载；或在系统设置中卸载。
- 签名密钥保存在项目 `keystore/aurora-release.jks`。**后续升级必须使用同一密钥**，否则只能卸载重装。

---

9. 版本

版本	日期	说明
1.0.7	2026-11	自检、自定义单灯、数量默认 1、3×100ms、动画延长
1.0.5	2026-11	颜色关闭、亮灯数量、只发不读、启动动画
1.0.3	2026-11	中科蓝讯 11 帧协议、BBB0 UUID、9 色、5×100ms、开发者模式、扫描状态修复
1.0.2	2026-11	UUID 改为 BBB0 系列，保留启动修复
1.0.1	2026-11	修复启动闪退，增加错误页
1.0.0	2026-11	首个版本：BLE 扫描/连接、开关、亮度、四种光效、速度、演示模式

中科蓝讯 RGB 灯控 BLE 协议规范

版本：2.6（基于你更新的 `RGB协议.xlsx`）
日期：2026-11
配套 App：幻彩灯控 v1.0.9

---

1. 角色与传输



- 手机是 GATT Client / Central，负责扫描、连接、写 Control 特征值。
- 灯控板是 GATT Server / Peripheral，负责广播、执行指令。
- App 使用 Write With Response 串行发送。

---

2. GATT 定义

名称	16 位 UUID	Android 完整 UUID	属性	说明
Service	`0xBBB0`	`0000BBB0-0000-1000-8000-00805F9B34FB`	Primary Service	灯控主服务
Control	`0xBBB1`	`0000BBB1-0000-1000-8000-00805F9B34FB`	Write、Write Without Response	手机写入控制帧
Status	`0xBBB2`	`0000BBB2-0000-1000-8000-00805F9B34FB`	Notify	预留状态/应答通道；当前 App 不依赖它
Info	`0xBBB3`	`0000BBB3-0000-1000-8000-00805F9B34FB`	Read	设备信息，可选
CCCD	`0x2902`	`00002902-0000-1000-8000-00805F9B34FB`	Write	Status 的 Notify 开关

- 这是中科蓝讯属性表里的标准 16 位 UUID；Android 会自动展开成 `0000xxxx-0000-1000-8000-00805F9B34FB`。
- **广播 BBB0 不是连接前提**，但建议广播，App 会把设备标记成“幻彩灯”并优先排序。

- 设备名不要求固定；App 会扫描所有 BLE 设备，用户手动选择。

---

3. 帧格式

```
偏移   0     1     2     3 .. 2+LEN   3+LEN
     0x11  CMD   LEN   DATA...        CHECKSUM
```

字段	长度	说明
Header	1	固定 `0x11`
CMD	1	命令字
LEN	1	DATA 字节数，0–32
DATA	LEN	命令参数
CHECKSUM	1	`(0x11 + CMD + LEN + DATA 所有字节) & 0xFF`

校验和

协议表只写了“检验和”，没有写算法。这里定义为**累加和取低 8 位**：

```
uint8_t checksum(const uint8_t *data, size_t len) {
    uint8_t sum = 0;
    for (size_t i = 0; i < len; i++) {
        sum = (uint8_t)(sum + data[i]);
    }
    return sum;
}
```

Java / Kotlin：

```
static int checksum(byte[] data, int length) {
    int sum = 0;
    for (int i = 0; i < length; i++) {
        sum = (sum + (data[i] & 0xFF)) & 0xFF;
    }
    return sum;
}
```

> 如果中科蓝讯原厂固件要求 XOR 或其它校验方式，请告诉我，我可以只改校验函数，其它帧结构不变。

---

4. 协议表已定义命令

4.1 开关

命令	帧	说明
开灯	`11 01 01 01 14`	DATA=01
关灯	`11 01 01 00 13`	DATA=00

4.2 光效模式

每个模式是独立命令，DATA=`01` 表示启用该模式，`00` 表示关闭该模式。

模式	CMD	开启帧	关闭帧	App 中的名字
跑马灯	`0x02`	`11 02 01 01 15`	`11 02 01 00 14`	流光跑马
呼吸	`0x03`	`11 03 01 01 16`	`11 03 01 00 15`	呼吸
火焰	`0x04`	`11 04 01 01 17`	`11 04 01 00 16`	火焰
律动	`0x05`	`11 05 01 01 18`	`11 05 01 00 17`	律动

短按模式：开启该模式（DATA=01）；再短按同一模式：关闭该模式（DATA=00），App 同时取消高亮。

4.3 灯光颜色

颜色	DATA	帧
红	`01`	`11 08 01 01 1B`
橙	`02`	`11 08 01 02 1C`
黄	`03`	`11 08 01 03 1D`
绿	`04`	`11 08 01 04 1E`
青	`05`	`11 08 01 05 1F`
蓝	`06`	`11 08 01 06 20`
紫	`07`	`11 08 01 07 21`
白	`08`	`11 08 01 08 22`
品红	`09`	`11 08 01 09 23`
关闭颜色	`00`	`11 08 01 00 1A`

App 会按这个顺序显示 9 个颜色圆点；再次点击当前颜色发送 DATA=00 关闭颜色并取消高亮。

4.4 亮度

命令	帧	范围
设置亮度	`11 10 01 xx CS`	`xx=0x00-0x64`（0–100）

示例：亮度 100% → `11 10 01 64 86`。

---

5. 补充命令

5.1 关闭光效模式

`0x06`，DATA=`01` / `00`：

- 关闭：`11 06 01 00 18`
- 恢复：`11 06 01 01 19`

App 的单个模式关闭使用对应模式命令 DATA=00；`0x06` 作为“关闭光效模式”的通用命令保留，供固件/调试使用。

5.2 一键关闭

`0x11 01 00`：

```
11 11 01 00 23
```

App 的“一键关闭”按钮发送这一帧，并同时把界面总开关置为关闭、所有高亮熄灭、亮度显示 0%。

5.3 自检

`0xFF`，DATA=`01` / `00`：

```
11 FF 01 01 12
```

5.4 亮灯数量

CMD	名称	DATA	说明
`0x07`	数量 +1/-1	`01` / `00`	加一颗：`11 07 01 01 1A`；减一颗：`11 07 01 00 19`

`0x07` 是**相对命令**，重复发送 3 次会加/减 3 颗，因此 App 每次点击只发 1 次，不做重复。

5.5 自定义单颗灯

`0x12`，按协议示例固定为：

```
11 12 02 x y CS
```

- `x`：灯序号 0–255；
- `y`：1 点亮，0 关闭；
- 第三字节固定 `02`（LEN=2 字节：序号 + 状态）。

示例：

操作	帧
点亮第 3 颗	`11 12 02 03 01 29`
关闭第 3 颗	`11 12 02 03 00 28`
点亮第 10 颗	`11 12 02 0A 01 30`
关闭第 10 颗	`11 12 02 0A 00 2F`
退出自定义/全部关闭	`11 12 02 00 00 25`

App 为每颗灯维护高亮状态；快速点击时按点击顺序排队，一组 3 次发完再发下一组。

5.6 App 扩展命令

CMD	名称	DATA	说明
`0x0A`	SPEED	`xx`	速度 0–100；示例 60 → `11 0A 01 3C 58`
`0x0B`	QUERY	无	查询当前状态；示例 `11 0B 00 1C`
`0x0F`	SYNC	6 字节	`[power, mode, color(0=关), brightness, speed, ledCount]`

`SYNC` 示例（开灯、流光、青色、亮度 72、速度 60、60 颗灯）：

```
11 0F 06 01 01 05 48 3C 3C ED
```

如果中科蓝讯固件不需要这些扩展，可以忽略；App 的主要控制仍使用协议表已定义的 01/02/03/04/05/07/08/10/11/12/FF。

---

6. App 的发送节奏

用户每次操作，App 默认把**同一个命令连续发送 3 次，间隔 100ms**：

```
t=0ms   第 1 帧
t=100ms 第 2 帧
t=200ms 第 3 帧
```

例外：

- `0x07` 是相对命令，每次只发 1 次。
- 自定义 `0x12` 每组 3 次，串行排队；不会跳过。
- 亮度拖动时每约 120ms 发 1 次当前值；松手后再发 3 次。
- 模式切换先发旧模式关闭 3 次，延迟 600ms 再发新模式开启 3 次。

App 连接成功后会先发一次 `SYNC`（3 次）。

---

7. 颜色/模式的完整映射

App 模式值	App 显示	协议 CMD	DATA
1	流光跑马	`0x02`	`01`
2	呼吸	`0x03`	`01`
3	律动	`0x05`	`01`
4	火焰	`0x04`	`01`
0	静态（选颜色后）	`0x08`	颜色码 1–9，或 0 关闭

---

8. nRF Connect 联调

1. 扫描并连接你的产品（名称任意）。
2. 展开 `0000BBB0-...` 服务。
3. 找到 `0000BBB1-...`（Control），点写入。
4. 选择 **Write**（带响应），输入： `11 01 01 01 14` 灯应开。
5. 输入 `11 08 01 01 1B` 切红色。
6. 输入 `11 03 01 01 16` 切呼吸。
7. App **不读取** `0000BBB2-...` 状态，也不写 CCCD；如设备有 Notify，可用 nRF Connect 自行开启观察原始字节。

---

9. 已知限制

- 协议表没有定义设备 → 手机的 ACK/状态帧，因此 App 采用“发送即认为成功”的乐观 UI。
- `Status` 已订阅，但当前只记录原始 Notify，不解析。
- 校验和算法是按“累加和”补充的；如果原厂固件用 XOR，需要同步改。
- 广播 BBB0、设备名都不是连接前提；真正必须的是 GATT 里的 BBB0/BBB1/BBB2。

---

10. 版本

协议版本	App 版本	说明
2.6	v1.0.9	GitHub 自动更新、上次设备自动连接；仅新增网络/安装权限，不改 BLE 帧
2.5	v1.0.8	开发者日志仅第三页、灯珠彩色按钮、连接后默认关灯
2.4	v1.0.7	首屏开关修复、自定义灯光大按钮/内置调色、0x12 长度 02
2.3	v1.0.6	自检 UI、0x12 自定义单灯、数量默认 1、3×100ms、启动动画延长
2.2	v1.0.5	颜色关闭、亮灯数量、只发不读、启动动画
2.1	v1.0.4	模式关闭、一键关闭、三页 UI、总开关前置
2.0	v1.0.3	`0x11` 帧、BBB0 UUID、9 色

ESP32 参考固件烧录指南

本指南用于把项目内的 `firmware/aurora_light_esp32/aurora_light_esp32.ino` 烧录到 ESP32，让灯带按 App 的自定义 BLE 协议工作。

> 已有成品灯带（厂商私有协议）不能直接套用本固件，除非你能拆开更换控制器或修改原固件。请先阅读《使用手册》第 5 节“设备兼容性”。

---

1. 你需要准备

硬件

- ESP32 开发板（ESP32-WROOM-32 / ESP32-DevKitC 等，带 BLE）
- WS2812B / SK6812 灯带（示例 60 颗）
- 5V 电源（电流 ≥ 灯珠数 × 60mA × 0.6，建议留余量）
- 电平转换/驱动（3.3V 数据 → 5V 灯带，可用 74HCT245 或 330Ω 串联电阻）
- 杜邦线、USB 数据线

接线

ESP32	灯带
GPIO 5（可改 `LED_PIN`）	DIN
GND	GND
5V（独立电源）	5V
GND（独立电源）	GND（必须与 ESP32 共地）

大功率灯带不要从 ESP32 的 5V 引脚取电，否则容易烧板或掉压重启。长灯带请每隔 2–3 米补电。

软件

- Arduino IDE 2.x
- ESP32 开发板支持包（Arduino-ESP32 **2.0.x**，例如 2.0.17）
- 库：
- **FastLED 3.6.x**
- **NimBLE-Arduino 1.4.3**（注意不要装 2.x，回调签名不同）

---

2. 安装 ESP32 开发板

1. 打开 Arduino IDE → File → Preferences。
2. 在 “Additional boards manager URLs” 填入：`https://espressif.github.io/arduino-esp32/package_esp32_index.json`
3. Tools → Board → Boards Manager，搜索 `esp32`，安装 **2.0.17**（或 2.0.x 系列）。
4. Tools → Board 选择 **ESP32 Dev Module**。
5. 端口选择 ESP32 对应的 COM 口。

---

3. 安装库

方式 A：库管理器

1. Sketch → Include Library → Manage Libraries。
2. 搜索 `FastLED`，安装 3.6.x。
3. 搜索 `NimBLE`，选择 **NimBLE-Arduino 1.4.3** 安装。 - 如果只能看到 2.x，请到 GitHub Releases 下载 1.4.3 的 ZIP，再用 Sketch → Include Library → Add .ZIP Library 安装。

方式 B：手动

把库解压到 `Documents/Arduino/libraries/`，确认目录结构：

```
libraries/
├─ FastLED/
└─ NimBLE-Arduino/
```

---

4. 修改参数

打开 `aurora_light_esp32.ino`，按你的硬件修改：

```
#define LED_PIN       5      // 数据线接的 GPIO
#define NUM_LEDS      60     // 灯珠数量
#define LED_TYPE      WS2812B
#define COLOR_ORDER   GRB    // 常见 WS2812B 是 GRB；颜色不对时改 RGB
#define MAX_MILLIAMPS 2500   // 电源限流，按实际电源调整
```

- 灯带颜色红绿颠倒：把 `COLOR_ORDER` 改为 `RGB` 或 `BGR`。
- 灯带闪烁/重启：降低 `MAX_MILLIAMPS`，检查共地，给 ESP32 单独稳压。
- 只有前几颗亮：`NUM_LEDS` 设大了或电源压降，检查补电。

---

5. 编译与烧录

1. 用 Arduino IDE 打开 `.ino`。
2. 点左上角“验证”（✓）编译。
3. 连接 ESP32，点“上传”（→）。
4. 打开 Serial Monitor，波特率 **115200**，复位板子，应看到：`AuroraLight ready: AuroraLight-3F7A` 名称后缀来自 ESP32 的蓝牙 MAC 后两字节。

常见编译错误

错误	原因	解决
`NimBLECharacteristicCallbacks::onWrite` 参数不匹配	装了 NimBLE 2.x	换回 1.4.3
`esp_read_mac` 未声明	缺少头文件	确认 `.ino` 中有 `#include <esp_mac.h>`
`FastLED` 找不到	库未安装	库管理器安装 FastLED
上传超时	没按 BOOT / 串口被占用	按住 BOOT 再点上传，关闭其他串口工具
灯不亮但串口正常	接线/颜色顺序/电源	检查 DIN、GND、5V、`COLOR_ORDER`

---

6. 验证 BLE

用 nRF Connect

1. 扫描，应看到 `AuroraLight-XXXX`。
2. 展开服务 `0000BBB0-0000-1000-8000-00805F9B34FB`。
3. 应看到： - Control：`0000BBB1-...`，可写 - Status：`0000BBB2-...`，Notify - Info：`0000BBB3-...`，只读
4. 向 Control 写入 `11 01 01 01 14`，灯应点亮。
5. 写入 `11 08 01 01 1B` 应切红色；`11 03 01 01 16` 应切呼吸。

用 App

1. 打开幻彩灯控，授予蓝牙权限。
2. 点“连接灯光”，选择 `AuroraLight-XXXX`。
3. 连接成功后测试开关、亮度、颜色、光效和速度。
4. 若 App 报“未找到自定义服务”，检查固件 UUID 是否被改动、是否成功启动 BLE。

7. 让灯效更好

- **颜色校准**：不同批次灯珠色偏，可在固件里对 RGB 做系数校正。
- **音乐律动**：当前固件用模拟节拍；要真实随音乐，可接 MAX9814/INMP441，读取包络并映射到亮度。
- **多灯同步**：多个 ESP32 各自作为从机，App 一次只能连一个；需要多灯同步时，可让一个主机 ESP32 通过 ESP-NOW 广播状态给其他从机。
- **断电记忆**：用 `Preferences` 库把状态写入 NVS，上电恢复。
- **安全**：BLE 不配对不加密，仅适合个人氛围灯；不要用于门锁等场景。

---

8. 参考

- 协议字段与示例帧：`BLE协议规范.md`
- App 构建与二次开发：`开发与构建说明.md`
- 完整固件：`../firmware/aurora_light_esp32/aurora_light_esp32.ino`

自动更新说明（GitHub Releases + Pages）

配套 App：v1.0.9 及以上
费用：0 元（使用 GitHub 免费额度）

---

1. 原理

```
App 启动
  → 请求 https://用户名.github.io/仓库名/update.json
  → 比较 versionCode
  → 有新版本：弹窗
  → 用户点“立即更新”
  → 下载 GitHub Release 里的 APK
  → 校验 SHA-256
  → 拉起系统安装界面
  → 用户点“安装”覆盖旧版
```

普通 App 无法静默安装，最后一步必须由用户确认。

---

2. 一次性配置

2.1 创建 GitHub 仓库

把本项目推送到 GitHub，例如：

```
https://github.com/12197096277-pixel/RGB_APP
```

2.2 开启 Pages

仓库 **Settings** → **Pages**：

- Source：`Deploy from a branch`
- Branch：`gh-pages`
- Folder：`/ (root)`

第一次发布后 `gh-pages` 分支才会出现。

2.3 配置 Secrets

仓库 **Settings** → **Secrets and variables** → **Actions**，新增：

Secret	内容
`KEYSTORE_BASE64`	`keystore/aurora-release.jks` 的 Base64
`KEYSTORE_PASSWORD`	keystore 密码
`KEY_ALIAS`	`aurora`
`KEY_PASSWORD`	key 密码

生成 Base64（PowerShell）：

```
[Convert]::ToBase64String([IO.File]::ReadAllBytes("keystore/aurora-release.jks")) | Set-Clipboard
```

> 不要把 keystore、密码、Base64 提交到公开仓库。

2.4 修改 App 更新地址

已配置为你的仓库：

```
AURORA_UPDATE_URL=https://l2197096277-pixel.github.io/RGB_APP/update.json
```

如果仓库改名，同步修改这里。

2.5 GitHub Actions 权限

仓库 **Settings → Actions → General → Workflow permissions**：

- 选择 **Read and write permissions**；
- 允许 GitHub Actions 创建 Release 和推送 `gh-pages`。

---

3. 发布新版本

1. 修改代码；
2. 提升 `app/build.gradle` 里的 `versionCode` 和 `versionName`；
3. 更新 `CHANGELOG.md`；
4. 提交并打 tag：

```
git tag v1.0.10
git push origin v1.0.10
```

或者 GitHub Actions 页面手动运行 `Release AuroraLight`。

Workflow 会自动：

- 构建签名 APK；
- 创建 GitHub Release；
- 上传 APK；
- 生成 `update.json`；
- 推送到 `gh-pages`。

App 下次启动就会看到更新弹窗。

---

4. update.json 格式

```
{
  "versionCode": 11,
  "versionName": "1.0.10",
  "apkUrl": "https://github.com/l2197096277-pixel/RGB_APP/releases/download/v1.0.10/AuroraLight-v1.0.10.apk",
  "sha256": "APK的SHA256",
  "changelog": "1. 修复...\n2. 新增...",
  "force": false
}
```

- `force=true`：不可取消，只能更新或退出；
- `force=false`：可以点“稍后”。

---

5. 手机端要求

- Android 8 及以上首次更新需要允许“安装未知应用”；
- App 会引导打开系统设置；
- 返回 App 后会自动继续安装；
- 必须使用同一个签名证书，否则系统拒绝覆盖安装。

---

6. 费用

- GitHub 仓库/Releases/Pages/Actions：免费额度内 0 元；
- 国内访问 GitHub 可能较慢；
- 如果以后用户量大、下载慢，可以只把 `update.json` 和 APK 换到阿里云 OSS，App 代码不用改，只改 URL。

开发与构建说明

版本：v1.0.9
包名：`com.aurora.light`
最低系统：Android 6.0（API 23）
目标系统：Android 15（API 35）

---

1. 技术选型

项目	选择	原因
语言	Java 8	无 Kotlin 运行时依赖，APK 小
UI	原生 Android View + 自绘 View	不引入 AndroidX / Material，安装包约 236 KB
BLE	Android 原生 `BluetoothLeScanner` / `BluetoothGatt`	直接、可控，无第三方 SDK
构建	Gradle 8.9 + Android Gradle Plugin 8.7.3	Android Studio 可直接打开、调试、运行；`build.ps1` 保留为无 Gradle 备用方案
数据	`SharedPreferences`	只保存本机偏好，无数据库、无账号
网络	无	清单中未声明 `INTERNET` 权限

没有使用：Firebase、广告 SDK、统计 SDK、账号系统、云端接口、WebView。

---

2. 项目结构

```
aurora-light/
├─ app/
│  ├─ build.gradle                  模块构建、版本号、签名
│  └─ src/
│     ├─ main/
│     │  ├─ AndroidManifest.xml
│     │  ├─ java/com/aurora/light/
│     │  │  ├─ MainActivity.java          主页面、状态机、交互
│     │  │  ├─ ScanSheet.java             底部扫描/权限面板
│     │  │  ├─ AuroraBleManager.java      扫描、连接、Notify、写队列
│     │  │  ├─ AuroraProtocol.java        中科蓝讯 11 帧、校验和、命令
│     │  │  ├─ AuroraState.java           灯光状态
│     │  │  ├─ BleDevice.java             扫描结果模型
│     │  │  ├─ AuroraApplication.java     崩溃兜底记录
│     │  │  ├─ CrashLog.java              启动错误保存
│     │  │  └─ ui/                        自绘 View
│     │  └─ res/                          布局、图标、颜色、动画
│     └─ test/java/.../AuroraProtocolTest.java   JUnit 协议测试
├─ gradle/wrapper/                 Gradle wrapper（含 jar）
├─ gradlew / gradlew.bat           命令行构建入口
├─ build.gradle                    根构建脚本
├─ settings.gradle                 工程与仓库配置
├─ gradle.properties               Gradle/Android 配置
├─ keystore.properties             签名配置（请勿公开）
├─ firmware/aurora_light_esp32/    ESP32 参考固件
├─ docs/                           使用手册、协议、开发文档
├─ tools/                          图标生成、dex 注入、预览页
├─ keystore/aurora-release.jks     项目签名密钥（务必保管）
├─ build.ps1                       无 Gradle 的备用构建
└─ dist/                           构建产物
```

---

3. 开发与构建

3.1 用 Android Studio 开发（推荐）

1. 安装 **Android Studio**（建议 Ladybug 2024.2 或更新版本，自带 JDK 17）。
2. 在 SDK Manager 安装 **Android 15 / API 35**。
3. `File → Open`，选择项目根目录 `aurora-light`。
4. 等待 Gradle Sync。首次会下载 Gradle 8.9、AGP 8.7.3 和 JUnit。
5. 手机开启 USB 调试并连接，点右上角 **Run ▶**；也可以先用演示模式看 UI。

Android Studio 会自动生成 `local.properties` 指向本机 SDK，不要提交到 Git。

> AGP 8 可能提示 `AndroidManifest.xml` 里的 `package` 属性已忽略；实际包名由 `app/build.gradle` 的 `namespace` / `applicationId` 决定。保留该属性是为了兼容无 Gradle 的 `build.ps1`，不影响构建。

Debug 和 Release 都使用项目签名（`keystore.properties` 指向 `keystore/aurora-release.jks`），所以：

- Android Studio 直接 Run 的 debug 包可以覆盖安装已发布的 APK；
- 改完代码不用卸载重装；
- 正式发包前只需要提升 `versionCode`。

3.2 命令行 Gradle

Windows：

```
.\gradlew.bat test
.\gradlew.bat assembleDebug
.\gradlew.bat assembleRelease
```

macOS / Linux：

```
./gradlew test
./gradlew assembleDebug
./gradlew assembleRelease
```

产物：

```
app/build/outputs/apk/debug/app-debug.apk
app/build/outputs/apk/release/app-release.apk
app/build/reports/tests/testDebugUnitTest/index.html
```

如果命令行没有设置 SDK，可先创建 `local.properties`：

```
sdk.dir=C\:\\你的路径\\Android\\Sdk
```

或设置环境变量 `ANDROID_HOME` / `ANDROID_SDK_ROOT`。

> 项目的 `gradle/wrapper/gradle-wrapper.properties` 使用腾讯镜像下载 Gradle 8.9，国内首次同步更快。如果镜像不可用，把 `distributionUrl` 改回：`https://services.gradle.org/distributions/gradle-8.9-bin.zip`。

3.3 版本号与签名

- 版本号在 `app/build.gradle`：

```
versionCode 7
versionName '1.0.9'
```

每次对外发版，把 `versionCode` 加 1，例如 `2`，`versionName` 改成 `1.1.0`。

- 签名配置在 `keystore.properties`：

```
storeFile=keystore/aurora-release.jks
storePassword=AuroraLight2026!
keyAlias=aurora
keyPassword=AuroraLight2026!
```

- 密钥别名：`aurora`
- 证书：RSA 2048，有效期 10000 天

重要：后续升级必须使用同一密钥。丢失密钥后，已安装用户只能卸载重装。正式发布前建议改成自己的强密码，并离线备份 JKS；不要把 `keystore.properties` 和 JKS 上传到公开仓库。

3.4 备用：无 Gradle 构建

适合没有 Android Studio、只想命令行出包的场景。依赖：

- JDK 17
- Android SDK：`platforms/android-35`、`build-tools/35.0.0`
- Python 3（仅用于把 `classes.dex` 写入 APK）

```
powershell -NoProfile -ExecutionPolicy Bypass -File .\build.ps1 `
  -Jdk "C:\Java\jdk-17" `
  -Sdk "C:\Android\Sdk" `
  -Python "C:\Python312\python.exe"
```

脚本会执行 `aapt2 compile/link → javac → d8 → 注入 dex → zipalign → apksigner`，输出 `dist/AuroraLight-v1.0.9.apk` 和 `dist/BUILD-INFO.txt`。

> Windows PowerShell 5.1 读取 UTF-8 中文脚本需要 BOM。仓库内的 `build.ps1` 已带 BOM；若重新保存后出现“字符串缺少终止符”，请保留 UTF-8 BOM。

3.5 安装

```
adb install -r app/build/outputs/apk/debug/app-debug.apk
```

或安装已签名的发布包：

```
adb install -r dist/AuroraLight-v1.0.9.apk
```

3.6 用 VS Code 开发

VS Code 适合改代码、跑 Gradle 任务、安装和看日志；它**不是完整的 Android IDE**，断点调试 App 仍建议用 Android Studio。

1. VS Code 打开项目根目录。
2. 安装推荐扩展（打开项目时会提示）： - Java Extension Pack - Gradle for Java
3. `Ctrl+Shift+B` 构建 Debug。
4. `Terminal → Run Task`： - `Android: Build Debug` - `Android: Build Release` - `Android: Test` - `Android: Build + Install + Launch` - `Android: Logcat` - `Android: Clean`
5. 任务通过 `tools/vscode-gradle.ps1` 自动寻找 JDK/SDK，并调用 `gradlew.bat`。

配置文件：

- `/.vscode/tasks.json`
- `/.vscode/extensions.json`
- `/.vscode/settings.json`

注意事项：

- 直接按 F5 不会启动 Android App；请用任务，或在 Android Studio 里 Run。
- 手机需要开启 USB 调试；`Build + Install + Launch` 会调用 `adb install -r` 和 `am start`。
- 如果自动探测失败，设置环境变量 `JAVA_HOME`（JDK 17）和 `ANDROID_HOME`（Android SDK 35）。
- 这套任务按 Windows PowerShell 编写；macOS/Linux 可直接在终端运行 `./gradlew ...`。

或把 APK 发到手机，用文件管理器点击安装（允许“安装未知应用”）。

---

4. 架构说明

4.1 状态流

```
用户操作
  │
  ├─ 更新 AuroraState（本地 UI 立即响应）
  ├─ saveState() 写入 SharedPreferences
  └─ 若已连接：AuroraProtocol 组帧 → sendBurst(3 次 / 100ms)
                                │
灯带 Notify ──────────────────▶ 原始字节日志（当前协议未定义状态帧）
```

设计原则：**UI 先响应，蓝牙异步发送。**未连接时允许预先设置，连接成功后 `SYNC` 一次性对齐。

4.2 `AuroraBleManager`

- 扫描：`BluetoothLeScanner`，无过滤扫描 12 秒，按 RSSI/是否广播服务排序由 `ScanSheet` 完成。
- 连接：`connectGatt(..., TRANSPORT_LE)`。
- 服务发现：查找 3 个 UUID；缺失则报“该设备不是幻彩灯”。
- 订阅：写 Status 的 CCCD；1.2 秒兜底标记 ready（部分固件不回 `onDescriptorWrite`）。
- MTU：CCCD 写成功后请求 64。
- 写队列：`ArrayDeque<byte[]>` + `WRITE_TYPE_DEFAULT`，收到 `onCharacteristicWrite` 才发下一条；2 秒超时保护。
- 通知：订阅 `0xBBB2`，当前只把原始 Notify 字节写入日志；协议尚未定义设备→手机状态帧。
- 线程：BLE 回调在 Binder 线程，统一 `Handler(Looper.getMainLooper())` 切主线程。
- 生命周期：`close()` 停止扫描、断开 GATT、清空回调。

4.3 协议实现

所有 UUID 和命令集中在 `AuroraProtocol`。修改 UUID 时必须同步修改：

1. `AuroraProtocol.java` 的三个 UUID；
2. ESP32 固件中的三个 `*_UUID` 字符串；
3. 文档中的 UUID 表；
4. 若改了设备名前缀，同步 `DEVICE_PREFIX` 与固件 `snprintf`。

4.4 UI 实现

- `AuroraBackgroundView`：深色纵向渐变 + 4 团 `RadialGradient` 柔光 + 星点 + 底部渐暗；仅在可见时 30fps 刷新。
- `PowerButtonView`：径向光晕 + `SweepGradient` 扫光弧 + 玻璃圆盘 + 手绘电源符号；关闭后停止刷新。
- `ModeTileView`：用 `StateListDrawable` 与 `ColorStateList` 表达选中态。
- `ColorDotView`：选中有白色描边和外发光。
- `GradientTextView`：用 `LinearGradient` 给文字上色。
- 所有布局使用 `dp` / `sp`，适配 360dp 宽度；主内容可滚动。

---

5. 权限

权限	适用版本	用途
`BLUETOOTH_SCAN`	Android 12+	扫描 BLE，声明 `neverForLocation`
`BLUETOOTH_CONNECT`	Android 12+	连接、读取设备名、GATT 操作
`ACCESS_FINE_LOCATION`	Android 6–11（maxSdk=30）	系统要求扫描 BLE 必须有定位权限
`BLUETOOTH` / `BLUETOOTH_ADMIN`	Android 6–11（maxSdk=30）	旧版蓝牙权限

- 未声明 `INTERNET`。
- 未声明后台定位、前台服务、通知权限。
- Android 11 及以下还必须打开系统定位开关；`ScanSheet` 会检测并引导。

---

6. 测试清单

6.1 静态验证与单元测试

```
# 协议 JUnit 测试（无需真机）
.\gradlew.bat test

# 签名
apksigner verify --verbose --print-certs app\build\outputs\apk\release\app-release.apk

# 清单
aapt2 dump badging app\build\outputs\apk\release\app-release.apk

# 确认没有 INTERNET 权限
aapt2 dump permissions app\build\outputs\apk\release\app-release.apk
```

测试报告：`app/build/reports/tests/testDebugUnitTest/index.html`。

6.2 安装与启动

```
adb install -r app/build/outputs/apk/debug/app-debug.apk
adb shell am start -n com.aurora.light/.MainActivity
adb logcat -s AuroraBle AndroidRuntime
```

6.3 功能检查

- [ ] 首次启动无登录页，直接进入主界面
- [ ] 底部按钮打开扫描面板

- [ ] 拒绝权限时显示权限引导，可进入演示模式
- [ ] 蓝牙关闭时显示“蓝牙未开启”，可跳转开启
- [ ] Android 11 及以下定位关闭时显示引导
- [ ] 演示模式下开关、亮度、模式、颜色、速度均有视觉反馈
- [ ] 连接真实/模拟从机后状态变为绿色“已连接”
- [ ] 连接后自动发送 SYNC + QUERY
- [ ] 断开后恢复“未连接”，设置保留
- [ ] 杀进程重开，设置保留
- [ ] 旋转保持竖屏（Activity 固定 portrait）
- [ ] 小屏（360×640dp）内容可滚动，无控件重叠

6.4 BLE 联调

用 nRF Connect 按《BLE 协议规范》第 9 节验证；或在 ESP32 上烧录参考固件，用 App 实际连接。

---

7. 已知限制

- **未在真实灯带上做硬件在环测试**：协议和固件代码已按规范实现，但不同灯带/控制器需按协议适配后验证。
- 只支持单设备连接；没有多灯分组、场景保存、定时、音乐频谱（需要麦克风数据）功能。
- 没有自动重连（系统回收/超出范围后会断开，需要手动重连）。
- BLE 连接不加密、不配对；不适合安全场景。
- 没有 OTA 固件升级。
- 颜色为 RGB 直接映射，未做色温/亮度校准。
- 扫描列表会显示附近所有 BLE 设备，非本协议设备连接时会报错；这是为了方便调试。
- App 只发送 30/60/90 三档速度；协议支持 0–100。

---

8. 二次开发建议

8.1 日常修改流程

1. **先备份/建分支**：至少复制一份 `aurora-light` 目录，或使用 Git。
2. 用 Android Studio 打开项目，等 Gradle Sync 完成。
3. 改代码后先跑 `.\gradlew.bat test`（协议逻辑改动时必跑）。
4. 连接手机点 **Run ▶**；纯 UI 改动可先用演示模式看效果。
5. 涉及 BLE 的改动，用真实灯带或 ESP32 参考固件联调。
6. 发版前提升 `versionCode`，执行 `assembleRelease`，并把新 APK 和文档一起归档。

8.2 常见修改对照

想改什么	App 侧	固件侧	备注
文字/颜色/图标	`res/values`、`res/drawable`、`res/mipmap`	不需要	改完直接 Run
预设颜色	`MainActivity.PRESET_COLORS`	不需要	RGB 范围不变
新光效	`AuroraProtocol` + `MainActivity`	`applyCommand` + `renderLeds`	协议模式表要同步更新
改 UUID/设备名	`AuroraProtocol`	UUID/广播名	必须一起升级，旧固件不兼容
改亮度范围	SeekBar/协议校验	亮度映射	当前统一 0–100
自动重连	`AuroraBleManager`	不需要	保存 MAC、断开后延时重连
多灯分组	`AuroraBleManager` + 新页面	不需要	需要改单连接模型
登录/云同步	新增加网络层和后端	不需要	会引入 `INTERNET` 权限，破坏离线设计

8.3 具体建议

1. **改 App 名称/图标**：修改 `strings.xml` 的 `app_name`，替换 `res/mipmap-*` 图标，重新构建。
2. **改 UUID**：见 4.3；改完必须重新烧录固件。
3. **加新光效**：在 `AuroraProtocol` 增加 CMD / MODE 常量，`MainActivity` 加卡片，固件 `applyCommand` 与 `renderLeds` 增加分支。
4. **加自动重连**：保存上次 MAC，在 `onBleDisconnected` 后延时 `connectGatt`；注意退出页面时取消。
5. **加音乐律动**：手机端采集麦克风需要 `RECORD_AUDIO`，或灯带端接 MAX9814/INMP441，把包络映射到亮度；当前固件是模拟节拍。
6. **正式发布**：更换签名密钥、改 `versionCode`、补隐私政策、在真机多机型验证、考虑上架应用商店的合规要求。

---

9. 构建产物

Gradle（推荐）

- `app/build/outputs/apk/debug/app-debug.apk`：可直接安装的调试包
- `app/build/outputs/apk/release/app-release.apk`：已签名发布包
- `app/build/reports/tests/testDebugUnitTest/index.html`：JUnit 测试报告

备用脚本

`dist/` 下：

- `AuroraLight-v1.0.9.apk`：已签名安装包
- `BUILD-INFO.txt`：版本、大小、SHA-256

`build/`、`app/build/` 是中间产物，可安全删除；`keystore/` 和 `keystore.properties` 必须保留。

更新日志

> 详细版按版本放在 `docs/changelogs/` 目录，每版一个文件；本文件是索引和摘要。
>
> 历史版本：v1.0.0–v1.0.8 均在，未删除。

v1.0.9 — 2026-11-13

新增

- GitHub Releases + Pages 自动更新：启动检查、弹窗、下载、SHA-256 校验、系统安装。
- 记住上次连接的灯带地址，下次启动自动连接。
- 新增 `.github/workflows/release.yml` 一键发布流程。
- 新增 INTERNET / REQUEST_INSTALL_PACKAGES / FileProvider。

详细说明：v1.0.9.md

---

v1.0.8 — 2026-11-08

修复/优化

- 开发者日志框只在“我的”页显示，切换页面自动隐藏。
- 自定义灯珠按钮加入按序号生成的彩色渐变，点亮更明显。
- 蓝牙连接成功后不再自动开灯，必须用户自己按总开关。
- 未连接时大开关显示“灯光未连接”。

详细说明：v1.0.8.md

---

v1.0.7 — 2026-11-08

修复/优化

- 修复未连接蓝牙时大开关误显示“灯光已开启”，改为“灯光未连接”并熄灭。
- 自定义灯珠改为 4 列大卡片，外观与主界面统一。
- 自定义灯光内置 9 色颜色条，选色作用于已点亮的单灯。
- 0x12 自定义单灯修正为 2 字节数据：`11 12 02 x y`。
- 退出自定义灯光发送 `11 12 02 00 00 25`。

详细说明：v1.0.7.md

---

v1.0.6 — 2026-11-08

新增/优化

- “光效”页新增自检按钮（`0xFF`）。
- 亮灯数量默认 1 个，`+` / `-` 直接发送 `11 07`，不再延迟/不再用 0x12 设置数量。
- 新增自定义灯光全屏界面：输入灯珠总数（1–255），生成 1…N 按钮，逐颗点亮/关闭。
- 自定义灯珠改为 4 列大卡片，内置 9 色颜色条。
- 修复未连接蓝牙时大开关误显示“灯光已开启”，改为“灯光未连接”。
- 自定义单灯发送 `11 12 02 x y`；退出发送 `11 12 02 00 00 25`。
- 自定义灯珠快速点击进入串行队列，不跳过。
- 普通指令改为连续 3 次 / 100ms。
- 启动动画延长到约 1.25 秒。

详细说明：v1.0.6.md

---

v1.0.5 — 2026-11-08

新增/优化

- 当前颜色再次点击会取消高亮，并发送 `11 08 01 00 1A`。
- 新增亮灯数量 `-` / `+` 和 0–255 滑块；颜色作用于当前数量。
- 未连接蓝牙时，总开关、亮度、模式、颜色、速度、数量、一键关闭都会提示先连接。
- 新增 RGB 灯带启动动画，约 0.9 秒。
- App 只发送指令，移除 Notify/CCCD/读取。
- 协议新增 `0x07` 数量加减、`0x12` 设置数量、`0x08 00` 关闭颜色；速度/查询扩展改为 `0x0A/0x0B`。

详细说明：v1.0.5.md

---

v1.0.4 — 2026-11-08

新增/优化

- 主界面拆成“控制 / 光效 / 我的”三页，支持左右滑动和底部切换。
- 总开关未开启时，亮度禁用并显示 0%，模式/颜色/速度高光熄灭；点击其它功能提示先开灯。
- 关闭总开关后所有高光熄灭、亮度归 0；重新开灯恢复上次亮度。

- 亮度拖动时每约 120ms 发一次当前值，松手后再补发 5 次。
- 模式短按开启、再按同一模式关闭并取消高亮；切换模式时先关旧模式。
- 颜色只能在光效关闭时选择；模式运行时会提示先关闭光效；选色成功提示颜色名。
- 新增“一键关闭”按钮，发送 `11 11 01 00 23`。
- 移除底部小灯条；开发者模式改为长按“我的”页面版本号进入，界面不再提示。
- 扫描超时缩短到 8 秒，使用更积极的扫描参数；修复断开后重新扫描状态。
- 协议新增 `0x06` 关闭光效、 `0x11` 一键关闭、 `0xFF` 自检；速度/查询扩展改为 `0x07/0x09`。

详细说明：v1.0.4.md

---

v1.0.3 — 2026-11-08

新功能

- 按 `RGB协议.xlsx` 改为中科蓝讯 `0x11` 帧协议，UUID 使用 `0xBBB0/0xBBB1/0xBBB2/0xBBB3`。
- 新增开发者模式：长按屏幕底部小灯条进入，再长按退出；同一页面内显示 App 实际发出的十六进制码，字号较小，可滚动、可清空。
- 每条控制指令连续发送 **5 次，间隔 100ms。**
- 颜色改为协议定义的 9 种：红、橙、黄、绿、青、蓝、紫、白、品红。
- 新增协议扩展命令： `0x06` 速度、 `0x07` 查询、 `0x0F` 全量同步。

修复

- 修复“连接过一次、断开后再次打开扫描面板，一个设备都看不到”的问题：
- 关闭扫描面板时重置内部扫描状态；
- 重新扫描前先停止旧扫描，避免 `SCAN_FAILED_ALREADY_STARTED` / 扫描频率限制；
- 扫描失败时保留错误提示，不再被“未发现设备”覆盖。
- 保留 v1.0.1 的启动崩溃修复：蓝牙初始化权限异常不再阻断启动；启动失败会显示可复制的错误页。

文档/固件

- 重写 BLE 协议文档，补充校验和算法、示例帧、9 色表、5×100ms 说明。
- ESP32 参考固件同步改为 `0xBBB0` + `0x11` 帧协议。
- 补充 VS Code 任务与 Android Studio 工程说明。

安装

- versionCode：4
- versionName：1.0.3
- 覆盖安装即可，签名与旧版一致。

---

v1.0.2 — 2026-11-08

- UUID 改为中科蓝讯 `0xBBB0/0xBBB1/0xBBB2/0xBBB3`。
- 保留启动崩溃修复。

v1.0.1 — 2026-11-08

- 修复启动闪退：蓝牙 `BluetoothAdapter` 初始化增加权限异常保护。
- 增加“启动失败”错误页，可复制错误信息。
- 移除启动期可能触发类加载问题的 API 30 Insets 代码。

v1.0.0 — 2026-11-08

- 首个版本：BLE 扫描/连接、开关、亮度、四种光效、六色、速度、演示模式、状态同步。
- 无注册、无登录、无联网权限。